

Virtual Worker Platform (VWP)

Prompt Library (JSON) + UI Templates (HTML/CSS/JavaScript)

Version: 1.0

MVP Domains: Finance, Technical Operations, Sales & Marketing, Logistics, Developer

PART A — PROMPT LIBRARY (JSON)

A1) How to Use

- Store these prompt objects in your DB or a `prompts/` folder.
 - Your AI Gateway selects prompts by `type` + `domain` + `template_id`.
 - Every AI call MUST include:
 - `system_prompt` (global rules)
 - `task_prompt` (classification/extraction/clarification/etc.)
 - `json_schema` (the ONLY valid output shape)
 - The model MUST respond with **valid JSON only**.
-

A2) Global Prompts (applies to all domains)

```
[
  {
    "id": "SYS_GLOBAL_001",
    "type": "system",
    "domain": "global",
    "name": "Global System Rules",
    "system_prompt": "You are the AI Interpretation Layer of the Virtual Worker Platform. Your job is to (1) classify a user request to a single approved template, (2) extract structured inputs that match a provided JSON Schema exactly, and (3) request clarification when required fields are missing or confidence is low. You must never execute actions, never access credentials, never invent fields, and never output text outside JSON. Return valid JSON only.",
    "json_schema": {
      "type": "object",
      "properties": {
        "status": { "type": "string", "enum": ["ok", "needs_clarification", "no_match"] },
        "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
      }
    }
  }
]
```

```

    "data": { "type": ["object", "null"] },
    "missing_fields": { "type": "array", "items": { "type": "string" } },
    "notes": { "type": "string" }
  },
  "required": ["status", "confidence", "data", "missing_fields", "notes"],
  "additionalProperties": false
}
},
{
  "id": "CLS_GLOBAL_001",
  "type": "classification",
  "domain": "global",
  "name": "Template Classification",
  "task_prompt": {
    "template": "User task: {user_input}\n\nAvailable templates (id, name,
description): {templates}\n\nSelect ONE best template id. If none apply, use
selected_template_id=null and status=no_match. Provide confidence 0..1.\nReturn
JSON only.",
    "variables": ["user_input", "templates"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "status": { "type": "string", "enum": ["ok", "no_match"] },
      "selected_template_id": { "type": ["string", "null"] },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
      "rationale": { "type": "string" }
    },
    "required": ["status", "selected_template_id", "confidence", "rationale"],
    "additionalProperties": false
  }
},
{
  "id": "CLARIFY_GLOBAL_001",
  "type": "clarification",
  "domain": "global",
  "name": "Ask Only For Missing Fields",
  "task_prompt": {
    "template": "User task: {user_input}\nSelected template: {template_id}
\nMissing required fields: {missing_fields}\n\nWrite a concise clarification
request asking ONLY for the missing fields. Return JSON only.",
    "variables": ["user_input", "template_id", "missing_fields"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "status": { "type": "string", "enum": ["needs_clarification"] },
      "questions": {

```

```

        "type": "array",
        "items": {
            "type": "object",
            "properties": {
                "field": { "type": "string" },
                "question": { "type": "string" },
                "example": { "type": "string" }
            },
            "required": ["field", "question", "example"],
            "additionalProperties": false
        }
    },
    "required": ["status", "questions"],
    "additionalProperties": false
},
{
    "id": "RISK_GLOBAL_001",
    "type": "risk_flagging",
    "domain": "global",
    "name": "Risk Flags From Inputs",
    "task_prompt": {
        "template": "Given: template_id={template_id} and
extracted_inputs={inputs}. Identify risk_flags from this list only:
[external_domain, bulk_operation, high_impact_system, pii_suspected]. Return
JSON only.",
        "variables": ["template_id", "inputs"]
    },
    "json_schema": {
        "type": "object",
        "properties": {
            "risk_flags": { "type": "array", "items": { "type": "string" } },
            "risk_summary": { "type": "string" }
        },
        "required": ["risk_flags", "risk_summary"],
        "additionalProperties": false
    }
}
]

```

A3) NEW: Job Description → Virtual Worker Role (Company “Worker Profiles”)

Companies can upload a Job Description (JD). The platform converts it into a **Worker Profile** that: - defines responsibilities and boundaries - maps responsibilities to approved templates - sets tone/style guidelines (how the worker communicates) - enforces hard safety constraints (cannot exceed platform governance)

A3.1 Worker Profile JSON Schema (stored per workspace)

```
{
  "worker_profile_schema": {
    "type": "object",
    "properties": {
      "worker_profile_id": { "type": "string" },
      "name": { "type": "string" },
      "role_title": { "type": "string" },
      "department": { "type": "string" },
      "seniority": { "type": "string", "enum": ["junior", "mid", "senior",
"lead"] },
      "responsibilities": { "type": "array", "items": { "type": "string" } },
      "allowed_domains": { "type": "array", "items": { "type": "string" } },
      "restricted_actions": { "type": "array", "items": { "type": "string" } },
      "preferred_templates": { "type": "array", "items": { "type": "string" } },
      "approval_rules_overrides": {
        "type": "object",
        "properties": {
          "always_require_approval_for": { "type": "array", "items": { "type":
"string" } },
          "never_require_approval_for": { "type": "array", "items": { "type":
"string" } }
        },
        "additionalProperties": false
      },
      "communication_style": {
        "type": "object",
        "properties": {
          "tone": { "type": "string", "enum": ["professional", "friendly",
"direct"] },
          "verbosity": { "type": "string", "enum": ["short", "medium",
"detailed"] },
          "formats": { "type": "array", "items": { "type": "string", "enum":
["email", "report", "bullet_summary"] } }
        },
        "required": ["tone", "verbosity", "formats"],
        "additionalProperties": false
      }
    }
  }
}
```

```

    },
    "required": ["name", "role_title", "responsibilities",
"preferred_templates", "communication_style"],
    "additionalProperties": false
  }
}

```

A3.2 JD-to-Profile Prompt (creates a Worker Profile)

```

{
  "id": "ROLE_FROM_JD_001",
  "type": "role_profile_generation",
  "domain": "global",
  "name": "Job Description → Worker Profile",
  "task_prompt": {
    "template":
"Job Description text: {jd_text}\n\nApproved templates available in this
workspace: {templates}\n\nCreate a Worker Profile that matches the JD.\nRules:
(1) map responsibilities to preferred_templates (only from {templates}), (2) do
not invent capabilities, (3) add restricted_actions if JD includes sensitive
activities, (4) keep communication_style suitable for business. Return JSON
only.",
    "variables": ["jd_text", "templates"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "status": { "type": "string", "enum": ["ok", "needs_clarification"] },
      "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
      "data": { "$ref": "worker_profile_schema" },
      "missing_fields": { "type": "array", "items": { "type": "string" } },
      "notes": { "type": "string" }
    },
    "required": ["status", "confidence", "data", "missing_fields", "notes"],
    "additionalProperties": false
  }
}

```

A3.3 Task Classification Using a Worker Profile

When a user selects a Worker Profile (e.g., “Finance Analyst”), the classifier should bias toward that profile’s `preferred_templates`.

```

{
  "id": "CLS_WITH_ROLE_001",

```

```

    "type": "classification",
    "domain": "global",
    "name": "Classification With Worker Profile Bias",
    "task_prompt": {
      "template": "User task: {user_input}\n\nWorker profile:
{worker_profile_json}\n\nAvailable templates: {templates}\n\nSelect ONE
template. Prefer templates listed in worker_profile.preferred_templates unless
clearly wrong. Return JSON only.",
      "variables": ["user_input", "worker_profile_json", "templates"]
    },
    "json_schema": {
      "type": "object",
      "properties": {
        "status": { "type": "string", "enum": ["ok", "no_match"] },
        "selected_template_id": { "type": ["string", "null"] },
        "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
        "rationale": { "type": "string" }
      },
      "required": ["status", "selected_template_id", "confidence", "rationale"],
      "additionalProperties": false
    }
  }
}

```

A3.4 Hard Safety Boundary (Non-negotiable)

Worker Profiles can: - influence tone, preferred templates, clarifications - add *stricter* approval requirements

Worker Profiles cannot: - bypass RBAC - bypass approval gates required by policy - enable disallowed actions or integrations

A4) Domain Prompt Packs (Extraction + Drafting)

Finance — Expense Summary (template_id: FIN_EXPENSE_SUMMARY)

```

{
  "id": "EXT_FIN_001",
  "type": "extraction",
  "domain": "finance",
  "template_id": "FIN_EXPENSE_SUMMARY",
  "name": "Extract Expense Summary Inputs",
  "task_prompt": {
    "template": "User task: {user_input}\nAttachments: {attachment_list}
\n\nExtract inputs matching schema exactly. If required fields missing, set
status=needs_clarification and list missing_fields. Return JSON only.",
    "variables": ["user_input", "attachment_list"]
  }
}

```

```

},
"json_schema": {
  "type": "object",
  "properties": {
    "status": { "type": "string", "enum": ["ok", "needs_clarification"] },
    "confidence": { "type": "number", "minimum": 0, "maximum": 1 },
    "data": {
      "type": ["object", "null"],
      "properties": {
        "date_range_start": { "type": "string", "description": "YYYY-MM-DD" },
        "date_range_end": { "type": "string", "description": "YYYY-MM-DD" },
        "receipt_ids": { "type": "array", "items": { "type": "string" } },
        "output_format": { "type": "string", "enum": ["pdf", "xlsx", "csv"] }
      },
      "required": ["date_range_start", "date_range_end", "receipt_ids",
"output_format"],
      "additionalProperties": false
    },
    "missing_fields": { "type": "array", "items": { "type": "string" } },
    "notes": { "type": "string" }
  },
  "required": ["status", "confidence", "data", "missing_fields", "notes"],
  "additionalProperties": false
}
}

```

Sales & Marketing — Draft Outreach Email (template_id: SM_DRAFT_OUTREACH_EMAIL)

```

{
  "id": "DRF_SM_001",
  "type": "drafting",
  "domain": "sales_marketing",
  "template_id": "SM_DRAFT_OUTREACH_EMAIL",
  "name": "Draft Outreach Email (Approval-Gated)",
  "task_prompt": {
    "template": "Inputs: {inputs_json}\n\nWrite a short, professional outreach
email. Do NOT claim the email was sent. Return JSON only.",
    "variables": ["inputs_json"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "subject": { "type": "string" },
      "body": { "type": "string" },
      "tone": { "type": "string", "enum": ["professional", "friendly",
"direct"] },

```

```

    "compliance_note": { "type": "string" }
  },
  "required": ["subject", "body", "tone", "compliance_note"],
  "additionalProperties": false
}
}

```

Technical Ops — Ticket Field Extraction (template_id: OPS_CREATE_TICKET)

```

{
  "id": "EXT_OPS_001",
  "type": "extraction",
  "domain": "technical_ops",
  "template_id": "OPS_CREATE_TICKET",
  "name": "Extract Ticket Fields",
  "task_prompt": {
    "template": "User task: {user_input}\nSignals/Logs: {signals}\n\nExtract Jira/ServiceNow ticket fields matching schema. Return JSON only.",
    "variables": ["user_input", "signals"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "status": { "type": "string", "enum": ["ok", "needs_clarification"] },
      "confidence": { "type": "number" },
      "data": {
        "type": ["object", "null"],
        "properties": {
          "summary": { "type": "string" },
          "description": { "type": "string" },
          "severity": { "type": "string", "enum": ["low", "medium", "high", "critical"] },
          "system": { "type": "string" },
          "labels": { "type": "array", "items": { "type": "string" } }
        }
      },
      "required": ["summary", "description", "severity", "system"],
      "additionalProperties": false
    },
    "missing_fields": { "type": "array", "items": { "type": "string" } },
    "notes": { "type": "string" }
  },
  "required": ["status", "confidence", "data", "missing_fields", "notes"],
  "additionalProperties": false
}
}

```


Logistics — Shipment Status Inputs (template_id: LOG_SHIPMENT_STATUS)

```
{
  "id": "EXT_LOG_001",
  "type": "extraction",
  "domain": "logistics",
  "template_id": "LOG_SHIPMENT_STATUS",
  "name": "Extract Shipment Status Inputs",
  "task_prompt": {
    "template": "User task: {user_input}\n\nExtract tracking_number and carrier if stated. If missing, ask clarification. Return JSON only.",
    "variables": ["user_input"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "status": { "type": "string", "enum": ["ok", "needs_clarification"] },
      "confidence": { "type": "number" },
      "data": {
        "type": ["object", "null"],
        "properties": {
          "tracking_number": { "type": "string" },
          "carrier": { "type": "string" }
        },
        "required": ["tracking_number"],
        "additionalProperties": false
      },
      "missing_fields": { "type": "array", "items": { "type": "string" } },
      "notes": { "type": "string" }
    },
    "required": ["status", "confidence", "data", "missing_fields", "notes"],
    "additionalProperties": false
  }
}
```

Developer — PR Summary Output (template_id: DEV_PR_SUMMARY)

```
{
  "id": "SUM_DEV_001",
  "type": "summarization",
  "domain": "developer",
  "template_id": "DEV_PR_SUMMARY",
  "name": "Summarize Pull Request",
  "task_prompt": {
    "template": "PR title: {pr_title}\nPR description: {pr_description}\nDiff summary: {diff}\n\nReturn a concise summary and risk assessment. JSON only.",
  }
}
```

```

    "variables": ["pr_title", "pr_description", "diff"]
  },
  "json_schema": {
    "type": "object",
    "properties": {
      "summary": { "type": "string" },
      "key_changes": { "type": "array", "items": { "type": "string" } },
      "files_changed": { "type": "integer", "minimum": 0 },
      "risk_level": { "type": "string", "enum": ["low", "medium", "high"] },
      "review_focus": { "type": "array", "items": { "type": "string" } }
    },
    "required": ["summary", "key_changes", "files_changed", "risk_level",
      "review_focus"],
    "additionalProperties": false
  }
}

```

A5) Prompt Versioning & Audit Fields (Recommended)

```

{
  "prompt_metadata_standard": {
    "prompt_id": "STRING",
    "version": "SEMMVER",
    "owner": "godswill",
    "last_updated": "ISO_DATETIME",
    "hash_sha256": "SHA256_OF_PROMPT_OBJECT",
    "used_in_templates": ["TEMPLATE_ID_1", "TEMPLATE_ID_2"],
    "notes": "short change notes"
  }
}

```

PART B — UI TEMPLATES (COMMON WEB LANGUAGE: HTML/CSS/JavaScript)

These templates are plain HTML + CSS + vanilla JavaScript so they can be embedded in any stack.

B1) UI Pages Included

- Login (placeholder)
- Task Inbox (list)

- Task Create (chat-like submit)
- Task Detail (timeline + outputs)
- Approvals Queue (approve/reject)
- Templates Library (list + version)
- **Worker Profiles (NEW)**: upload job description, generate worker profile, manage profiles, select active worker for tasks

B2) Common API Helper (api.js)

```
<script>
  // api.js
  const API_BASE = "/api/v1";

  async function api(path, { method = "GET", body, headers = {} } = {}) {
    const res = await fetch(`${API_BASE}${path}`, {
      method,
      headers: {
        "Content-Type": "application/json",
        ...headers
      },
      body: body ? JSON.stringify(body) : undefined
    });

    const data = await res.json().catch(() => ({}));
    if (!res.ok) throw new Error(data?.error?.message || `API error $
{res.status}`);
    return data;
  }
</script>
```

B3) Minimal Styling (styles.css)

```
<style>
  :root { --bg:#0b1220; --card:#111a2e; --text:#e8eefc; --muted:#a7b4d4; --
accent:#3b82f6; --danger:#ef4444; --ok:#22c55e; }
  body { margin:0; font-family: ui-sans-serif, system-ui, -apple-system, Segoe
UI, Roboto, Arial; background:var(--bg); color:var(--text); }
  .wrap { max-width: 1100px; margin: 0 auto; padding: 16px; }
  .row { display:flex; gap:16px; }
  .col { flex:1; }
  .card { background:var(--card); border:1px solid rgba(255,255,255,0.08);
border-radius:14px; padding:14px; }
```

```

.muted { color:var(--muted); }
.btn { background:var(--accent); color:white; border:0; padding:10px 12px;
border-radius:10px; cursor:pointer; }
.btn.secondary { background:transparent; border:1px solid rgba(255,255,255,
0.15); }
.btn.danger { background:var(--danger); }
.btn.ok { background:var(--ok); }
.input, textarea { width:100%; padding:10px; border-radius:10px; border:1px
solid rgba(255,255,255,0.15); background:rgba(255,255,255,0.03); color:var(--
text); }
.list { display:flex; flex-direction:column; gap:10px; }
.item { padding:12px; border-radius:12px; border:1px solid rgba(255,255,255,
0.08); background:rgba(255,255,255,0.02); }
.tag { display:inline-block; padding:3px 8px; border-radius:999px; font-size:
12px; border:1px solid rgba(255,255,255,0.14); margin-left:8px; }
.grid2 { display:grid; grid-template-columns: 1fr 1fr; gap:12px; }
@media (max-width: 900px) { .row { flex-direction:column; } .grid2 { grid-
template-columns: 1fr; } }
</style>

```

B4) Task Inbox + Create (single page example)

```

<div class="wrap">
  <h1>Virtual Worker</h1>
  <p class="muted">Submit a task in plain language. The platform will classify,
request clarification if needed, and execute via approved workflows.</p>

  <div class="row">
    <div class="col card">
      <h2>Create Task</h2>
      <textarea id="taskDesc" rows="6" placeholder="e.g., Summarize these
receipts for February and produce an expense report as PDF."></textarea>
      <div style="display:flex; gap:10px; margin-top:10px;">
        <button class="btn" onclick="createTask()">Submit</button>
        <button class="btn secondary"
onclick="document.getElementById('taskDesc').value=''">Clear</button>
      </div>
      <p id="createMsg" class="muted" style="margin-top:10px;"></p>
    </div>

    <div class="col card">
      <h2>Task Inbox</h2>
      <div id="taskList" class="list"></div>
    </div>
  </div>

```

```

    </div>
</div>

<script>
  async function loadTasks() {
    const res = await api('/tasks');
    const tasks = res.data || [];
    const el = document.getElementById('taskList');
    el.innerHTML = '';

    if (!tasks.length) {
      el.innerHTML = '<div class="item muted">No tasks yet.</div>';
      return;
    }

    tasks.forEach(t => {
      const div = document.createElement('div');
      div.className = 'item';
      div.innerHTML = `
        <div style="display:flex; justify-content:space-between; gap:10px;">
          <div>
            <strong>${t.title || 'Task'}</strong>
            <span class="tag">${t.status}</span>
            ${t.template ? `<span class="tag">${t.template}</span>` : ''}
          </div>
          <button class="btn secondary" onclick="openTask('${t.id}')">Open</
button>
        </div>
        <div class="muted" style="margin-top:6px;">${(t.description ||
'').slice(0,120)}${(t.description||'').length>120?'...':''}</div>
      `;
      el.appendChild(div);
    });
  }

  async function createTask() {
    const desc = document.getElementById('taskDesc').value.trim();
    if (!desc) return;
    const msg = document.getElementById('createMsg');
    msg.textContent = 'Submitting...';
    try {
      await api('/tasks', { method: 'POST', body: { description: desc, title:
'New Task' } });
      msg.textContent = 'Submitted.';
      document.getElementById('taskDesc').value = '';
      await loadTasks();
    } catch (e) {
      msg.textContent = e.message;
    }
  }

```

```

    }
  }

  function openTask(id) {
    // Replace with routing. For MVP: navigate to /task.html?id=...
    window.location.href = `/task.html?id=${encodeURIComponent(id)}`;
  }

  loadTasks();
</script>

```

B5) Approvals Queue (approve/reject)

```

<div class="wrap card">
  <h2>Approvals</h2>
  <p class="muted">Sensitive actions pause here. Approvers decide; the workflow
  resumes or aborts.</p>
  <div id="approvalList" class="list"></div>
</div>

<script>
  async function loadApprovals() {
    const res = await api('/approvals?status=pending');
    const approvals = res.data || [];
    const el = document.getElementById('approvalList');
    el.innerHTML = '';

    if (!approvals.length) {
      el.innerHTML = '<div class="item muted">No pending approvals.</div>';
      return;
    }

    approvals.forEach(a => {
      const div = document.createElement('div');
      div.className = 'item';
      div.innerHTML = `
        <div><strong>${a.scope}</strong> <span class="tag">${a.status}</span></div>
        <div class="muted" style="margin-top:6px;">${a.summary} || 'Review
        required.'</div>
        <div style="display:flex; gap:10px; margin-top:10px;">
          <button class="btn ok" onclick="approve('${a.id}')">Approve</button>
          <button class="btn danger" onclick="reject('${a.id}')">Reject</button>
        </div>
      `;
    });
  }

```

```

        `;
        el.appendChild(div);
    });
}

async function approve(id) {
    await api(`/approvals/${id}/approve`, { method: 'POST', body: { notes:
'Approved' } });
    loadApprovals();
}

async function reject(id) {
    await api(`/approvals/${id}/reject`, { method: 'POST', body: { notes:
'Rejected' } });
    loadApprovals();
}

loadApprovals();
</script>

```

PART C — UI TEXT (COMMON LANGUAGE COPY)

Use consistent language across screens:

```

{
  "copy": {
    "task_submit_hint": "Describe what you want done. The Virtual Worker will
match an approved workflow and ask for missing info if needed.",
    "approval_hint": "This task includes a sensitive action. Please approve to
continue or reject to stop execution.",
    "audit_hint": "Every action is logged for accountability and compliance.",
    "low_confidence_hint":
"I'm not fully confident. Please choose a workflow or provide the missing
details."
  }
}

```

END